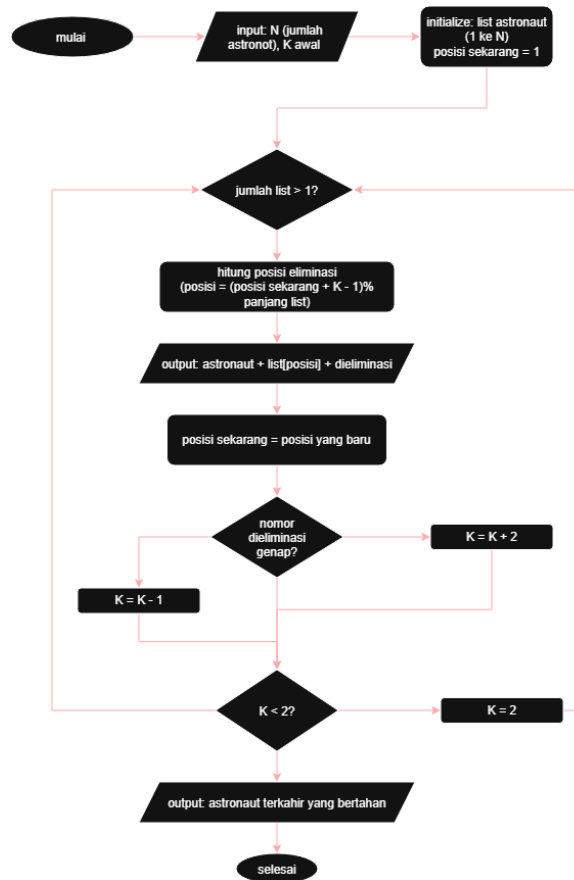


1. Problem 1: "The Last Astronaut"



"The Last Astronaut" menguji logika pemrosesan data melingkar (circular array) berbasis eliminasi bertahap. Dalam skenario ini, sebanyak N astronot berbaris membentuk lingkaran dan dieliminasi satu per satu berdasarkan nilai hitungan K yang bersifat dinamis. Keunikannya terletak pada aturan penyesuaian nilai K setelah setiap eliminasi: jika nomor astronot yang gugur bernilai genap, nilai K bertambah 2, sedangkan jika bernilai ganjil, nilai K berkurang 1. Selain itu, terdapat batas minimum untuk menjaga agar K tidak pernah kurang dari 2, sehingga iterasi eliminasi dapat terus berjalan secara konsisten hingga tersisa satu astronot terakhir.

Flowchart yang dirancang menggambarkan alur algoritma secara terstruktur mulai dari inisialisasi daftar astronot dan penentuan posisi awal hitungan. Pada setiap iterasi perulangan, posisi astronot yang tereliminasi dihitung menggunakan operasi aritmatika agar penelusuran tetap berputar di dalam lingkaran. Setelah astronot tersebut dihapus dari daftar dan nomornya dicetak, sistem melakukan percabangan kondisi untuk memeriksa sifat bilangan (genap atau ganjil) untuk memperbarui nilai K , serta mengevaluasi kondisi $K < 2$ untuk memastikan nilainya tetap valid. Alur perulangan ini terus mengecilkan ukuran daftar astronot sampai kondisi batas terlampaui (tersisa satu astronot), di mana sistem akan menampilkan pemenang akhir dan menghentikan proses.

2. Problem 2: Aline-In-The-Middle

PROGRAM Enkripsi_Alien_In_The_Middle

DEKLARASI:

```
    pesan_asli    : STRING
    pesan_sandi   : STRING
    i, panjang    : INTEGER
    nilai_curr     : INTEGER    // Nilai alfabet huruf sekarang
(1-26)
    nilai_prev    : INTEGER    // Nilai alfabet huruf sebelumnya
(1-26)
    nilai_baru     : INTEGER    // Nilai alfabet setelah digeser
```

ALGORITMA:

```
    BACA pesan_asli
    panjang <- PANJANG(pesan_asli)
    pesan_sandi <- ""

    JIKA panjang > 0 MAKA
        // Huruf pertama tidak mengalami perubahan pergeseran
        pesan_sandi <- pesan_sandi + pesan_asli[0]

        // Iterasi dari huruf kedua sampai akhir
        FOR i <- 1 TO panjang - 1 DO
            // Konversi huruf ke nilai alfabet (A=1, B=2, ...,
(1-26)
            nilai_curr <- ORD(pesan_asli[i]) - ORD('A') + 1
            nilai_prev <- ORD(pesan_asli[i - 1]) - ORD('A') +
1
            // Hitung pergeseran nilai baru
            nilai_baru <- nilai_curr + nilai_prev

            // Jika nilai melewati 26 (Z), lakukan wrapping
            kembali ke A
            JIKA nilai_baru > 26 MAKA
                nilai_baru <- (nilai_baru - 1) MOD 26 + 1
            ENDIF

            // Konversi kembali nilai alfabet ke karakter
huruf
```

```

                pesan_sandi <- pesan_sandi + CHAR(nilai_baru +
ORD('A') - 1)
            ENDFOR
        ENDIF

        TAMPILKAN pesan_sandi
    END PROGRAM

```

Pseudocode untuk permasalahan "Alien-In-The-Middle" ini dirancang untuk mensimulasikan mekanisme enkripsi teks berbasis pergeseran posisi alfabet relatif. Proses diawali dengan membaca string input dan mempertahankan huruf pertama tanpa perubahan, karena huruf pertama tidak memiliki karakter pendahulu sebagai acuan pergeseran. Selanjutnya, algoritma melakukan perulangan mulai dari karakter kedua hingga akhir pesan, di mana setiap huruf dikonversi ke dalam nilai numerik berbasis posisi alfabet (A=1 hingga Z=26). Nilai posisi huruf saat ini kemudian ditambahkan dengan nilai posisi huruf yang berada tepat di sebelumnya pada teks asli untuk menghasilkan nilai karakter baru.

Untuk menangani kasus ketika hasil penjumlahan nilai alfabet melebihi batas maksimum (Z=26), algoritma menerapkan operasi aritmatika modular agar urutan hitungan dapat berputar kembali ke awal alfabet (A). Setelah nilai numerik baru diperoleh, sistem mengonversinya kembali menjadi karakter huruf dan menggabungkannya ke dalam string hasil enkripsi. Keseluruhan alur ini memastikan pesan asli berhasil ditransformasikan ke dalam format sandi secara konsisten sesuai aturan pergeseran berantai sebelum akhirnya ditampilkan kepada pengguna.